

SHEETLINK — a SAS/R helper library for driving Smartsheet (no AI)

What it is. A small, deterministic library that lets a biostatistician keep a Smartsheet program tracker current **automatically from scheduled SAS or R** — no copy-paste, no manual status updates, no LLM. SAS/R writes the *truthful operational status* to Smartsheet by a stable key; **Smartsheet's own built-in workflows send the notifications.** That split is the whole design: *code owns the data, Smartsheet owns the alerts.*

```
validated nightly extract → SAS/R (this library) → Smartsheet cells (by key)
                                     |
                                     Smartsheet Automation (configured once)
                                     ▼
                                email / mobile push / Slack / Teams
```

Files

File	Role
ss_macros.sas	The SAS library. Every macro is %SS_*. PROC HTTP + JSON LIBNAME engine.
ss_companion.R	The R equivalent (httr2 + jsonlite). Same governance, same idempotency.
ss_config.sas	Per-sheet constants: sheet id, key column, recipients, the ops-only allowlist .
tracker_update.sas	The nightly driver you schedule. Build data → %ss_upsert → (attach) → heartbeat.

The five things this library guarantees

- Idempotent — never duplicates.** %ss_upsert / ss_upsert () match on a ****stable external key you own**** (a Task/Deliverable/Milestone code). If the key is on the sheet it **updates that row in place** (PUT); if not, it **appends** (POST). Run it ten times a night — the sheet converges to the same state. *Never blind-append.*
- Ops-only — no PHI ever leaves the tenant.** %ss_guard / ss_guard () check ****every column you are about to write against an allowlist**** (CFG_ALLOW). Try to push a non-allowlisted column and the job **fails loud and stops** — a coded boundary, not a habit. Allowed: milestone dates, deliverable/QC status, % complete, aggregate counts, ownership. **Never:** participant-level, unblinded, PHI, or reported clinical numbers.
- Token is never hard-coded or logged.** Read at runtime from SMARTSHEET_TOKEN (or a perms-600 file), inside OPTIONS NOSOURCE/req_auth_bearer_token so it never reaches the log. Rotate it on the Smartsheet side; the code never changes.
- Rate-limit aware.** Smartsheet allows ~300 requests/min/token. %ss_http / req_retry () retry **429 and 5xx with exponential backoff** and otherwise **fail loud**.
- Fails loud, never silent.** A stale tracker that *looks* fresh is the dangerous failure. %ss_assert emails CFG_BACKUP and aborts on any non-2xx, and the driver ends with a

heartbeat so silence is unambiguously a failure.

Quick start (SAS)

```
/* once: export your token in the scheduler's environment, NOT in code */
/* export SMARTSHEET_TOKEN=xxxxxxx (or store in a perms-600 file) */

%include "ss_config.sas"; /* set CFG_SHEET, CFG_KEYCOLNAME, CFG_ALLOW ... */
%include "ss_macros.sas";
%ss_init(tokenenv=&CFG_TOKEN_ENV, backup=&CFG_BACKUP, allow=&CFG_ALLOW);

data tracker;
  length TaskID $40 Status $20;
  TaskID="CP101-DBL"; Status="At Risk"; PctDone=0.60; output;
  label Status="Status" PctDone="% Done"; /* each label = the Smartsheet column TITLE */
run;

%ss_upsert(sheet=&CFG_SHEET, keycol=TaskID, keycolname=&CFG_KEYCOLNAME, data=tracker);
```

The **contract for %ss_upsert**: your dataset has the key variable plus one variable per

column to write, and **each value variable is labelled with its exact Smartsheet column

title** (label Status="Status" PctDone="% Done";). The macro reads those labels, resolves

each to a columnId from a freshly-fetched column map, JSON-escapes values, and splits the

work into a PUT (existing keys) and a POST (new keys).

Quick start (R)

```
source("ss_companion.R")
allow <- c("Task ID", "Status", "% Done", "Owner", "Due")
df <- tibble::tibble(`Task ID` = c("CP101-DBL", "CP101-ADaM"),
                    Status      = c("At Risk", "On Track"),
                    `% Done`    = c(0.60, 0.85))
ss_upsert(sheet = Sys.getenv("CP101_TRACKER_ID"), key = "Task ID", df = df, allow = allow)
```

In R the **data-frame column names are the Smartsheet titles** (back-tick names with spaces).

Macro / function reference

SAS	R	Does
%ss_init	ss_req (implicit)	Load token (hidden), base URL, allowlist, fail-loud target.
%ss_http	ss_req	One authenticated call; 429/5xx retry + backoff; fail loud on non-2xx.
%ss_colmap	ss_colmap	GET sheet → fresh column title → columnId map (rebuilt every run; ids are the only stable address).
%ss_get_rows	ss_get_rows	GET sheet rows → long (rowId, columnId, value); the key→rowId map.
%ss_guard	ss_guard	Enforce the ops-only allowlist ; fail loud on any non-allowlisted column.
%ss_upsert	ss_upsert	Idempotent update-by-key / add-if-new. The flagship.
%ss_attach	ss_attach	Attach an ops-only status PDF to a row (rate-intensive; once per run).
%ss_update_request	ss_update_request	Supported human-in-the-loop ask (emails a link to update rows).

SAS	R	Does
%ss_assert	ss_assert	Fail loud + notify backup + abort.

Notifications: configure once in Smartsheet, not in code (Pattern A)

The robust pattern is **not** to send emails from SAS. It is:

1. SAS/R writes the truthful cell (e.g. `Status = "At Risk"`).
2. A Smartsheet **Automation** you set up once — `*When Status changes to At Risk → Alert the PM / send a mobile push / post to a Teams channel*` — fires.

This keeps recipients, escalation, quiet-hours, and channels in Smartsheet (where a PM can edit them) and keeps your code about *data*. Use `%ss_update_request` only for a targeted "please confirm X" ask that needs a specific person to act.

Governance (the boundary that lets this be automated at all)

- **Smartsheet is a tracker, not a system of record.** The validated SAS/R outputs remain the source of truth; the sheet is a convenience mirror of *operational* status.
- **Allowlist is code, not a guideline.** Widening it is a reviewed change to `CFG_ALLOW`.
- **Aggregate only.** Counts and statuses, never a value attributable to one participant.
- Token in env/secret store, rotated, never logged. Least-privilege service account.

A note on robustness

The JSON LIBNAME engine names its child tables from the response structure

(`COLUMNS`, `ROWS`, `ROWS_CELLS`). If your SAS version names them differently, run a GET once and `PROC DATASETS lib=_ssj;` to confirm, then adjust `%ss_colmap/%ss_get_rows`.

Make the key a TEXT column. The idempotent match is on the key value as text on both sides (`cats( )`), so the Smartsheet key column should be a **Text/Number column holding text codes** (CP101-DBL), not an auto-number or a numeric column — otherwise a zero-padded code (007) can fail to match its own prior row and a duplicate is appended. Use a stable code you own as the key.

Validate the library against a **non-production sandbox sheet** before scheduling it against a live program tracker — the same way you'd validate any macro that writes to a shared artifact.